

Session Report — Deployments & Frontends

Date: May 29, 2026 **Scope:** All work done after initial 140 reports were generated **Author:** AI-assisted development session

1. GitHub Repository Setup

Two new repositories were created and pushed to GitHub under `voicenotesite`:

Repository	URL	Description
portfolio-daemon	GitHub	Background service manager CLI
ai-chat-proxy	GitHub	OpenAI API proxy (FastAPI)
reports	GitHub	140 AI-generated reports (PDF+MD)

All 7 repositories were made public.

2. Render.com Deployments

Three standalone services deployed on Render (Frankfurt region, free tier):

2.1 LinkShort — URL Shortener

Attribute	Value
URL	<code>https://fastapiurl.onrender.com</code>
Repo	<code>voicenotesite/FastAPI-url</code>
Stack	FastAPI + SQLAlchemy + SQLite
Frontend	React 19 SPA (dark theme, login, dashboard, copy-to-clipboard)

Attribute	Value
Auth	JWT + bcrypt (passlib)
Features	Create short URL, redirect, click stats, list, delete, is_active toggle
Tests	3 integration tests (pytest + httpx)
Deployment	Dockerfile + Render

2.2 GraphQL Blog

Attribute	Value
URL	https://graphql-blog-lxjy.onrender.com
Repo	voicenotesite/graphql-blog
Stack	FastAPI + Strawberry GraphQL + SQLAlchemy + SQLite
Auth	JWT
Features	Post CRUD, comments, user registration, GraphQL playground at <code>/graphql</code>
Tests	pytest + httpx

2.3 AI Chat Proxy

Attribute	Value
URL	https://ai-chat-proxy-twj4.onrender.com
Repo	voicenotesite/ai-chat-proxy
Stack	FastAPI + httpx + OpenAI API
Frontend	Custom stunning chat UI (see section 4)
Features	OpenAI proxy with streaming SSE, rate limiting (60 req/min), usage logging, custom API key support
Health	<code>/health</code>

2.4 Python Portfolio (unified API)

Attribute	Value
URL	https://python-portfolio-y0z8.onrender.com

Attribute	Value
Repo	voicenotesite/python-portfolio
Stack	FastAPI + SQLAlchemy + SQLite
Modules	Shortener, Blog (GraphQL), Chat, Queue, RAG
Frontend	New unified SPA (see section 3)

3. Unified Frontend (python-portfolio)

A single-page application serving Chat, Queue, and RAG modules under one roof.

URL: <https://python-portfolio-y0z8.onrender.com>

3.1 Chat Tab

- Model selector (GPT-3.5 Turbo, GPT-4)
- Message history with user/assistant bubbles
- API calls to `/api/chat/message`

3.2 Queue Tab

- Create tasks with type selector (Email Campaign, Generowanie Raportu, Eksport Danych, Generic)
- Real-time WebSocket progress updates (`wss://.../api/queue/ws/{id}`)
- Progress bars with percentage
- Cancel and clear-all controls
- Live task counter badge

3.3 RAG Tab

- PDF upload with drag & drop
- Document list with chunk count and delete
- Question input with TF-IDF vector search
- Confidence score with color coding
- Source file display

3.4 Tech Stack

- Vanilla JavaScript (no framework)

- CSS custom properties (dark theme)
 - WebSocket for real-time queue updates
 - Fetch API for REST endpoints
-

4. AI Chat Proxy Frontend

A standalone, production-quality chat interface.

URL: <https://ai-chat-proxy-twj4.onrender.com>

Features

Feature	Description
Streaming	Real-time SSE response rendering
Model Selector	GPT-4, GPT-3.5 Turbo, Claude 3 Sonnet
Live Status	Green/amber/red dot with health checks every 30s
Suggestions	4 one-click prompt hints on empty state
Settings Panel	Optional custom API key (localStorage)
Animations	Fade-in messages, typing indicators, glow effects
Design	Dark theme, glassmorphism header, Inter font
Responsive	Mobile-friendly layout

Architecture

- Self-contained HTML (inline CSS + JS)
 - Served directly by FastAPI backend
 - Relative API calls (same-origin proxy)
 - Rate limit warnings from server
-

5. Reports Site

URL: <https://voicenotesite.github.io/reports/>

Two versions: - **Dynamic** (/) — JS-based SPA with project cards and expandable report lists - **Static** (/nodynamic/) — Pure HTML, fully readable by AI crawlers (Gemini)

Contents

- 7 project directories × 20 reports = 140 reports
- Each report in PDF + Markdown format
- Index page with links to all reports
- robots.txt allowing all crawlers

Report Categories (per project)

1. Project Overview
 2. Architecture Analysis
 3. API Endpoints
 4. Database Schema
 5. Tech Stack & Dependencies
 6. Authentication & Authorization
 7. Frontend Analysis
 8. Configuration & Environment
 9. Security Audit
 10. Performance Analysis
 11. Code Quality Assessment
 12. Test Coverage Report
 13. Deployment Guide
 14. Error Handling Review
 15. Data Flow Analysis
 16. Module Dependencies
 17. Integration Points
 18. Known Issues & Bugs
 19. Improvement Recommendations
 20. Changelog & History
-

6. WebBartosz Portfolio Updates

URL: <https://voicenotesite.github.io/WebBartosz/>

Changes Made

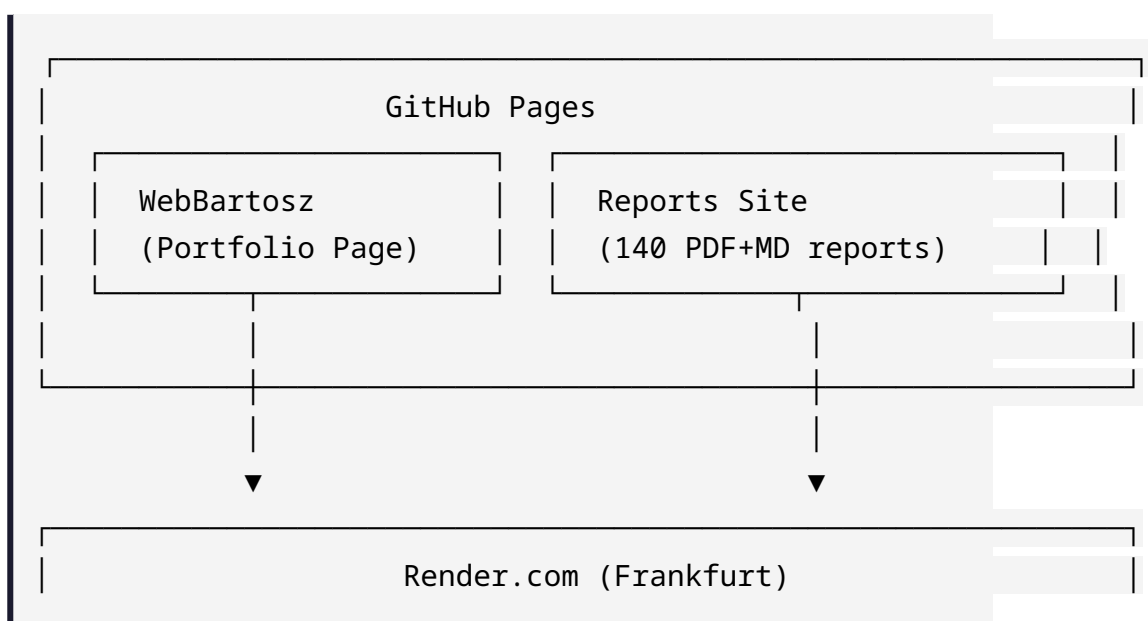
Change	Before	After
Project count	5/5	8/8

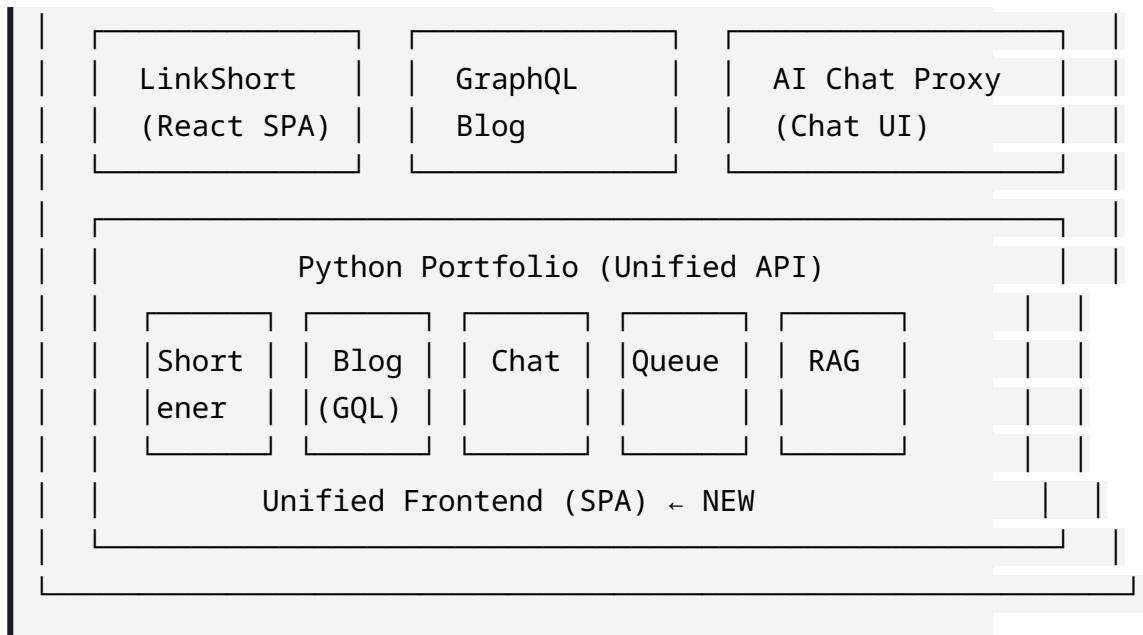
Change	Before	After
URL Shortener link	Swagger docs → <code>python-portfolio/docs</code>	Standalone frontend → <code>fastapiurl.onrender.com</code>
GraphQL Blog link	Unified API <code>/api/blog</code>	Standalone → <code>graphql-blog-lxjy.onrender.com/graphql</code>
AI Chat link	Unified API <code>/docs#/chat</code>	Standalone frontend → <code>ai-chat-proxy-twj4.onrender.com</code>
Queue/RAG links	Swagger docs sections	Unified frontend → <code>python-portfolio-y0z8.onrender.com</code>
Status bar	5 module checks	6 service checks + daemon CLI indicator
About section	"5 projektów w 1 API"	Updated links to all standalone services
Reports card	Missing	Added with links to dynamic + static versions

Live Status Monitoring

- 6 services checked every 60 seconds via health endpoints
- Color-coded dots (green = online, red = offline, gray = CLI)
- Overall status indicator in the status bar

7. Architecture Overview





8. Key Metrics

Metric	Value
Total repositories	7 (all public)
Render services	4 (all on free tier)
Frontends built	3 (unified SPA, AI Chat UI, Reports site)
Lines of frontend code	~800 (HTML + CSS + JS)
GitHub Pages sites	3 (WebBartosz, Reports, Reports/nodynamic)
Live API endpoints	~25 across all services
Real-time features	WebSocket queue updates, SSE chat streaming
Deployment region	Frankfurt (Render) + GitHub Pages CDN

9. Future Improvements

1. **Task Queue & RAG frontends** could be extracted into standalone services
2. **CI/CD pipeline** via GitHub Actions for automated testing
3. **PostgreSQL migration** for production-ready data persistence
4. **Custom domains** for Render services
5. **Authentication** across all modules (currently only LinkShort has JWT)

6. **Rate limiting** on unified API endpoints